



សាកលវិទ្យាល័យភូមិន្ទភ្នំពេញ
ROYAL UNIVERSITY OF PHNOM PENH

**ប្រព័ន្ធបំប្លែងអក្សរខ្មែរពីរូបភាពឡើងឯកសារ PDF
ដោយប្រើប្រាស់ប្រព័ន្ធ Tesseract-Base**

**System for converting Khmer text From images and PDF documents
using a Tesseract-based**

A Research Report
In Partial Fulfillment of the Requirements for the Degree of
Master of Science in Information Technology Engineering

Seang Sony

June 2026

សាកលវិទ្យាល័យភូមិន្ទភ្នំពេញ

ROYAL UNIVERSITY OF PHNOM PENH

ប្រព័ន្ធបំប្លែងអក្សរខ្មែរពីរូបភាពឡើងឯកសារ PDF

ដោយប្រើប្រាស់ប្រព័ន្ធ Tesseract-Base

**System for converting Khmer text From images and PDF documents
using a Tesseract-based**

A Research Report

**In Partial Fulfillment of the Requirements for the Degree of
Master of Science in Information Technology Engineering**

Seang Sony

Examination committee: DR. Mao Makara

June 2026

SUPERVISOR'S RESEARCH SUPERVISION STATEMENT

TO WHOM IT MAY CONCERN

Name of program: Master of Science in Information Technology Engineering

Name of candidate: SEANG SONY

Title of research report: System for converting Khmer text From images and PDF documents using a Tesseract-based

This is to certify that the research carried out for the above titled Bachelor's research report was completed by the above-named candidate under my direct supervision. This report material has not been used for any other degree. I played the following part in the preparation of this research report: System for converting Khmer text From images and PDF documents using a Tesseract-based

Supervisor's Name: DR. Mao Makara

Supervisor's signature:.....

Date.....

មូលដ្ឋានសង្ខេប

នៅក្នុងអត្ថបទស្រាវជ្រាវនេះ យើងនឹងបង្ហាញអំពីប្រព័ន្ធដំណើរការ OCR (Post-Processing Pipeline) ដែលត្រូវបានរចនាឡើងដើម្បីធ្វើឲ្យប្រសើរឡើងនូវគុណភាពនៃអត្ថបទដែលបានដកស្រង់ចេញពីឯកសារ PDF ភាសាខ្មែរ។ ប្រព័ន្ធ Tesseract OCR ដែលជាម៉ូដែល OCR open-source ដ៏ល្បីល្បាញ នៅតែផ្តល់នូវលទ្ធផលដែលមានកំហុសជាច្រើននៅពេលប្រើប្រាស់ជាមួយឯកសារភាសាខ្មែរដោយសារភាពស្មុគស្មាញនៃស្រ្តីបខ្មែរដូចជាព្យញ្ជនៈជាន់គ្នាស្រុះដែលមានទម្រង់ខុសៗគ្នាព្រមទាំងការដកឃ្លាដែលដែលជាចំណុចលំបាកសម្រាប់ប្រព័ន្ធ OCR ណាក៏ដោយ។

ការស្រាវជ្រាវនេះបានបង្កើតប្រព័ន្ធដំណើរការប្តូរដំណាក់កាលៗដំណាក់កាលទីមួយគឺការស្វែងរកស្វ័យប្រវត្តិនូវប្រភេទគុណភាពរូបភាព ដោយប្រើប្រាស់ការវិភាគ Laplacian variance, កម្រិតពន្លឺ និង white pixel ratio ដើម្បីបាត់បង់រូបភាពឯកសារជាបីប្រភេទ គឺ PDF ឌីជីថល ឯកសារស្កេន និងឯកសាររូបថត។ ដំណាក់កាលទីពីរ គឺការដំណើរការរូបភាពដោយប្រើប្រាស់បច្ចេកទេស computer vision ដូចជា bilateral filtering, CLAHE, gamma correction, binarization, deskewing និង morphological cleanup ដើម្បីបង្កើនភាពច្បាស់លាស់នៃអក្សរសម្រាប់ Tesseract OCR។ ដំណាក់កាលទីបី គឺការអនុវត្ត rule-based post-processing ដែលមានលើសពី ៦០ ចំណុចកែតម្រូវ ដើម្បីជួសជុលកំហុស OCR ដែលកើតឡើងជាប្រចាំ ដូចជាការបំប្លែងព្យញ្ជនៈខ្មែរ ការដកយកអត្ថបទ garbage ចេញ ។ ដំណាក់កាលទីបួន គឺការអនុវត្ត AI correction ដោយប្រើ large language model (Claude by Anthropic) ដើម្បីដោះស្រាយកំហុសស្មុគស្មាញ។

ប្រព័ន្ធទាំងមូលត្រូវបានអភិវឌ្ឍជាអ្នកប្រើ web application ដោយប្រើ Flask Python backend និង React frontend ដែលអនុញ្ញាតឲ្យអ្នកប្រើប្រាស់ដំណើរការឯកសារ PDF ភាសាខ្មែរដោយមិនចាំបាច់មានចំណេះដឹងផ្នែកកម្មវិធី។

ការស្រាវជ្រាវនេះបញ្ជាក់ថា ការរួមបញ្ចូលគ្នារវាងបច្ចេកទេស computer vision, rule-based correction និង AI language model គឺជាវិធីសាស្ត្រប្រកបដោយប្រសិទ្ធភាពខ្ពស់ក្នុងការដំណើរការ OCR ភាសាខ្មែរ ដោយមិនចាំបាច់ប្រើ GPU ឬប្រព័ន្ធ hardware ថ្លៃដើម្បីធ្វើការ inference។

ABSTRACT

Optical Character Recognition (OCR) for the Khmer language remains an unsolved practical problem. Tesseract, the most widely available open-source OCR engine with Khmer support, produces output containing numerous systematic errors when applied to real-world Khmer PDF documents, including misread vowels and diacritics, consonant substitutions, misidentification of the Khmer sentence-ending period (្ក) as the Latin digit 4, and garbage text introduced by logos, stamps, and form decorations. These errors prevent the digitization of government records, educational materials, and historical documents that exist only in scanned or photographed form.

This research designs, implements, and evaluates a complete post-processing pipeline that significantly improves Khmer OCR accuracy without requiring the retraining of recognition models. The proposed system organizes the extraction process into four phases: (1) automatic document quality detection followed by an adaptive image preprocessing pipeline that handles clean PDFs, scanned documents, and photographs differently; (2) Tesseract OCR text extraction; (3) a rule-based post-processing module containing more than sixty correction patterns organized into sequential steps for garbage removal, character substitution, symbol cleanup, punctuation correction, and broken word repair; and (4) an optional AI correction layer based on a large language model (Claude) for complex contextual errors beyond the reach of deterministic rules.

The complete system is delivered as a web application with a React frontend and a Flask backend, allowing non-technical users to upload Khmer PDF documents and receive corrected digital text within minutes. The results demonstrate that adaptive preprocessing combined with linguistic rule-based correction substantially improves Khmer OCR accuracy on real-world documents.

Keywords: Khmer OCR, Tesseract, post-processing, rule-based correction, large language models, web application, character error rate.

CANDIDATE'S STATEMENT

TO WHOM IT MAY CONCERN

This is to certify that the research report that I, Seang Sony, hereby present, entitled " System for converting Khmer text From images and PDF documents using a Tesseract-based," for the degree of Engineering in Information Technology at Royal University of Phnom Penh, is entirely my own work. Furthermore, it has not been used to fulfill the requirements of any other qualification, in the whole or in part, at this or any other University or equivalent institution. The research methodology, implementation, and findings represent original contributions to the field of Khmer document digitization, particularly in developing a comprehensive post-processing pipeline that combines automatic image quality detection, rule-based linguistic correction, and AI-assisted error refinement using large language models. Through this work, I have demonstrated strong research capabilities and independence in addressing the practical challenges of Khmer text extraction from real-world documents.

No reference to, or quotation from, this document may be made without the written approval of the author.

Name of Candidate: Seang Sony

Signed by the candidate:

Date:

Name of Supervisor: Dr. Mao Makara

Countersigned by the Supervisor:

Date:

ACKNOWLEDGMENTS

I would like to begin by expressing my sincere gratitude for the opportunity to pursue this research. I am deeply thankful to the Royal University of Phnom Penh for offering such a well-structured academic program within the Faculty of Engineering, which has provided a strong foundation for this success. The comprehensive curriculum, practical exposure, and academic environment have been instrumental in shaping my journey as both an engineer and a researcher.

I am especially grateful to all the faculty members for their dedicated teaching and continuous support throughout my studies. Their commitment to student learning has inspired and guided me significantly. I would also like to thank my classmates and fellow researchers for their collaboration, encouragement, and the shared experiences that made this journey memorable.

My deepest appreciation goes to my supervisor, Dr. Mao Makara, whose expertise, mentorship, and unwavering support have been vital to the completion of this work. He has only been an exceptional lecturer and supervisor but also a strong supporter at every stage of this research. His insights into Khmer language technology and document processing greatly shaped the direction of this project. This research would not have been possible without his guidance and patience throughout the development process.

Finally, I want to express my heartfelt thanks to my family for their financial support, constant encouragement, and unconditional love. Their sacrifices and belief in me have been the driving force behind my achievements. I am deeply grateful for their patience during the long hours spent on this research, and for always standing by my side throughout this academic journey.

TABLE OF CONTENTS

SUPERVISOR'S RESEARCH SUPERVISION STATEMENT	i
មូលដ្ឋានសង្ខេប	ii
ABSTRACT	iii
CANDIDATE'S STATEMENT	iv
ACKNOWLEDGMENTS	v
TABLE OF CONTENTS	vi
LIST OF TABLE	viii
LIST OF FIGURES	ix
CHAPTER 1 INTRODUCTION	10
1.1 Background to the Study	10
1.2 Problem Statement	10
1.3 Aims and Objectives	11
1.4 Rationale of the Study	12
1.5 Limitations and Scope	12
1.6 Research Contributions	12
CHAPTER 2 LITERATURE REVIEW	14
2.1 Khmer Script Overview	14
2.2 Optical Character Recognition Technology	14
2.3 Tesseract OCR for Low-Resource Languages	14
2.4 Image Preprocessing for OCR Quality Improvement	14
2.5 Rule-Based Post-Processing for OCR Error Correction	15
2.6 Large Language Models for Text Correction	15
2.7 Web-Based Document Processing Applications	15
2.8 Summary and Research Gap	16
CHAPTER 3 METHODOLOGY	17
3.1 Research Design	17
3.2 Overall System Architecture	17
3.3 Document Quality Detection	18
3.4 Tesseract OCR Configuration	19
3.5 AI Correction Integration	19
3.6 Web Application Architecture	20
3.7 Evaluation Methodology	21
CHAPTER 4 POST-PROCESSING IMPLEMENTATION	22
4.1 Overview	22
4.2 Step 0: Form Blank-Field Cleanup	22
4.3 Step 1: Line-Level Garbage Removal	23
4.4 Step 1.5: Inline Garbage Token Removal	23
4.5 Step 2: Khmer Character Corrections	23
4.6 Step 3: Symbol Cleanup	24
4.7 Khmer Punctuation Correction	24
4.8 Step 5: Broken Word Repair	24

4.9	AI Language Model Integration.....	24
4.10	Word Confidence Analysis and Highlighting.....	25
CHAPTER 5 SYSTEM DEVELOPMENT AND IMPLEMENTATION		26
5.1	Overview of the Web Application	26
5.2	File Upload Interface	26
5.3	Real-Time Processing Progress	27
5.4	Extraction Results Interface.....	27
5.5	Batch Processing	28
5.6	System Integration	29
CHAPTER 6 RESULTS AND DISCUSSION		30
6.1	Experimental Setup.....	30
6.2	Document Quality Detection Results.....	31
6.3	OCR Accuracy Improvement Results.....	31
6.4	Analysis of Post-Processing Step Contributions.....	33
6.5	AI Language Model Correction Analysis	34
6.6	Word Confidence Highlighting Evaluation	35
6.7	Web Application Performance.....	35
6.8	Comparison with Related Systems	36
CHAPTER 7 CONCLUSION.....		38
7.1	Summary of the Research	38
7.2	Summary of Key Findings	38
7.3	Limitations	38
7.4	Future Work	39
7.5	Final Remarks	39
REFERENCES		40

LIST OF TABLE

Table 1 Character-Level Corrections.....	Error! Bookmark not defined.
Table 2 Character Error Rate (CER) by Document Type and System Configuration.....	32
Table 3 Word Error Rate (WER) by Document Type and System Configuration	32
Table 4 Post-Processing Step Contribution Analysis	33
Table 5 Categories of AI-Corrected Errors.....	34
Table 6 Average Processing Time per Five-Page Document	36
Table 7 CER Comparison Between Tesseract Baseline and the Proposed Pipeline..	Error! Bookmark not defined.

LIST OF FIGURES

Figure 1 Overall System Architecture	18
Figure 2 Web Application Architecture.....	20
Figure 3 Post-Processing Pipeline	22
Figure 4 Error Recognition	25
Figure 5 Overall Interface of the Proposed System	26
Figure 6 File Upload Interface	26
Figure 7 Real-Time Processing.....	27
Figure 8 Extraction Results Interface	27
Figure 9 Download Options (.txt and .docx)	28
Figure 10 Extraction History Interface	28
Figure 11 Batch Processing Interface	29
Figure 12 OCR Accuracy Improvement before vs. after post-processing (CER/WER bar chart)	31
Figure 13 Per-Step Contribution to Accuracy relative impact of each post-processing step.	33
Figure 14 Word Confidence Highlighting Example.....	35
Figure 15 Web Application Screenshots upload page and results view.	36

CHAPTER 1

INTRODUCTION

1.1 Background to the Study

Optical Character Recognition (OCR) technology is a process that converts images of printed or scanned text into machine-readable digital text. Over the past two decades, OCR technology has become an essential tool for digitizing documents across many industries, including education, government administration, libraries, and cultural preservation. For widely used languages such as English, Chinese, and Japanese, OCR systems achieve high accuracy because large, annotated datasets and decades of research support them. However, for the Khmer language, which is the official language of Cambodia, OCR technology remains significantly underdeveloped. Khmer script has a uniquely complex structure that makes it particularly difficult for standard OCR engines to process accurately. Unlike Latin-based scripts where characters are placed sequentially from left to right on a single baseline, Khmer characters can stack vertically and combine with diacritics, subscripts, and vowel markers that appear above, below, or wrapped around the base character. A single missing or incorrect diacritic mark can completely change the meaning of a word. This structural complexity, combined with the limited availability of high-quality Khmer training data, means that even modern OCR engines such as Tesseract frequently produce errors when processing Khmer documents.

In Cambodia today, a large volume of important documents, including government records, university textbooks, legal contracts, scholarship forms, and historical materials, exist only as physical paper documents or low-quality scanned PDFs. The inability to accurately convert these documents into digital text creates significant barriers to education, research, and public administration. There is therefore a strong practical need for a reliable Khmer OCR system that can be used by non-technical users without requiring any programming knowledge. This project addresses this need by developing a complete Khmer OCR extraction system that combines intelligent image preprocessing, Tesseract OCR, rule-based post-processing corrections, and optional AI-powered text correction. The system is delivered as a web application that allows users to upload a PDF file and receive clean, corrected Khmer text within minutes.

1.2 Problem Statement

The core challenge of this project is that Tesseract OCR, while being the most widely available open-source OCR engine with Khmer language support, produces output that contains numerous and consistent types of errors when applied to real Khmer documents. These errors fall into several categories:

- **Image quality variation:** Real-world Khmer documents come in many physical conditions clean digital PDFs, flatbed scans with uneven lighting, and photographs taken with mobile cameras in poor lighting. A single fixed preprocessing approach cannot handle all these cases optimally.

Applying heavy noise-removal filtering to a clean digital PDF degrades the quality of thin character strokes, while applying no preprocessing to a dark scanned document results in characters merging together.

- **Khmer character misreads:** Tesseract frequently misidentifies specific Khmer characters due to their visual similarity. Common examples include missing vowel marks such as ្ក (u), ្ខ (uu), and ្គ (om), incorrect diacritics, and consonant confusions such as រ misread as វ, or វ misread as ឃ. These errors occur consistently across different documents and can be corrected through targeted pattern matching.
- **Garbage text from document decorations:** Scanned Khmer documents often contain logos, stamps, borders, dotted form fields, and page numbers that Tesseract reads as text. This produces lines of meaningless characters, sequences of dots and dashes, and fragments of Latin characters mixed into the Khmer output.
- **Khmer period misidentification:** The Khmer sentence-ending period ។ (Unicode U+17D4) is visually similar to the Latin digit 4 in many Khmer fonts. Tesseract consistently misreads this character as the numeral 4, causing all sentence boundaries in the extracted text to appear as numbers.
- **Complex errors requiring language understanding:** some errors cannot be fixed by simple pattern matching because they depend on understanding the meaning and context of the surrounding text. These require a language model that understands Khmer vocabulary and grammar.

1.3 Aims and Objectives

The aim of this project is to design, implement, and deploy a functional web-based Khmer OCR post-processing system that significantly improves the quality of text extracted from Khmer PDF documents compared to using Tesseract alone.

The specific objectives are:

- To design an automatic document quality detection algorithm that classifies input images as clean digital PDFs, scanned documents, or photographs, and selects the appropriate preprocessing pipeline for each type.
- To implement a multi-stage image preprocessing pipeline using computer vision techniques that optimizes document images for Tesseract OCR processing.
- To build a comprehensive rule-based post-processing module that corrects the most common and consistent Khmer OCR errors through pattern matching and character substitution.

- To integrate the AI language model Claude by Anthropic to handle complex errors that cannot be solved by rules alone.
- To develop a user-friendly web application with a React frontend and Flask backend that allows non-technical users to upload, process, and download corrected Khmer text.
- To evaluate the system's accuracy improvement using standard OCR metrics, specifically Character Error Rate (CER) and Word Error Rate (WER).

1.4 Rationale of the Study

This project is important for several practical and academic reasons. First, Cambodia is currently undergoing rapid digital transformation, and many critical documents still exist only in paper or scanned form. A reliable Khmer OCR tool would allow organizations to digitize these documents efficiently without requiring manual re-typing. Second, while Tesseract provides basic Khmer OCR capability, no widely available post-processing tool exists that specifically targets the consistent error patterns found in Khmer documents. This project fills that gap. Third, the system demonstrates that a combination of traditional computer vision preprocessing, linguistic rule-based correction, and modern AI language models can work together effectively even for low-resource languages.

1.5 Limitations and Scope

This project focuses specifically on extracting text from PDF documents that contain typeset Khmer text. The system does not handle handwritten Khmer text, which requires a fundamentally different approach involving handwriting recognition models. The AI correction feature requires an active internet connection and a valid Anthropic API key. The rule-based correction dictionary covers the most common error patterns found in Cambodian government documents, educational materials, and financial documents, but it cannot cover every possible Khmer word or every type of document. The evaluation in this project uses CER and WER metrics on a limited set of test documents.

1.6 Research Contributions

This research makes the following specific contributions to the field of Khmer document digitization and OCR technology:

- **Contribution 1:** Automatic document quality classification for Khmer images. This is the first documented system that automatically classifies Khmer document images into quality categories using measurable image properties and selects preprocessing parameters accordingly, eliminating manual document type selection.

- **Contribution 2: Combined rule-based and AI correction methodology.** This research demonstrates a practical methodology for combining deterministic rule-based correction (for high-frequency, precisely defined errors) with LLM-based correction (for complex, context-dependent errors) in a complementary pipeline. The two approaches address fundamentally different error categories and achieve greater combined accuracy than either approach alone.
- **Contribution 3: Accessible web application for non-technical users.** The system is delivered as a publicly deployable web application that makes advanced Khmer OCR capabilities accessible without requiring programming knowledge, GPU hardware, or software installation, addressing the practical usability gap identified as a limitation of prior research models.

CHAPTER 2

LITERATURE REVIEW

2.1 Khmer Script Overview

The Khmer script is an abugida used as the official writing system of Cambodia [1]. Each Khmer syllable is composed of a base consonant modified by dependent or independent vowels, diacritics, and subscript consonants placed above, below, or around the base. The Khmer Unicode block (U+1780–U+17FF) contains 33 consonants, 16 dependent vowels, 14 independent vowels, and 13 diacritics [2].

Two features make Khmer challenging for OCR: the coeng character (U+17D2) renders the following consonant as a small subscript that is visually distinct from its normal form, and Khmer text does not use inter-word spaces, creating segmentation challenges [3].

2.2 Optical Character Recognition Technology

Optical Character Recognition (OCR) converts images of text into machine-readable digital text. Modern systems rely on CNNs and transformer-based models [4] and achieve near-human accuracy on widely supported languages where large annotated datasets exist.

TrOCR [4] combines a Vision Transformer encoder [5] with a BART-style decoder, achieving state-of-the-art performance on English benchmarks. Its tokenizer does not include Khmer characters, limiting direct applicability without customization [6].

2.3 Tesseract OCR for Low-Resource Languages

Tesseract is an open-source OCR engine maintained by Google. Since version 4.0 it uses an LSTM neural network and ships with a trained Khmer model [7], but several studies report that its Khmer accuracy remains poor compared to Latin-based languages.

Measured a Character Error Rate (CER) of approximately 35.9% for unprocessed Tesseract on Khmer government documents, attributed to insufficient training data, the complexity of Khmer stacking characters, and Tesseract's Latin-script design assumptions. Fine-tuning on Khmer datasets has yielded moderate improvements but does not overcome these architectural limits [8].

2.4 Image Preprocessing for OCR Quality Improvement

Image preprocessing is critical because input image quality directly determines recognition accuracy. Liao et al. [9] showed that binarization, noise removal, and contrast enhancement consistently improve OCR accuracy across languages and document types; standard operations include grayscale conversion, adaptive thresholding, noise filtering, deskewing, and contrast normalization.

Adaptive thresholding based on local statistics outperforms global methods such as Otsu's on documents with uneven illumination [10]. Bilateral filtering provides edge-

preserving noise reduction by smoothing background noise while preserving character-stroke boundaries [11]. CLAHE corrects spatially varying illumination without amplifying noise in low-contrast regions [12].

Khmer preprocessing must preserve very fine details – vowel marks and diacritics can be as narrow as one or two pixels at standard scanning resolution, so aggressive noise removal destroys distinguishing features. Upscaling before noise removal and binarization mitigates this by providing more pixels per stroke [3].

2.5 Rule-Based Post-Processing for OCR Error Correction

Rule-based post-processing improves OCR output by applying deterministic correction rules derived from common error patterns. Systematic errors specific to a language or document type can be identified by analyzing OCR output and corrected through pattern matching and character substitution [13].

For Khmer OCR, several consistent error patterns have been documented. The Khmer sentence-ending period (U+17D4) is visually similar to the Latin digit 4 in many Khmer fonts, causing Tesseract to misread sentence boundaries as the number 4 consistently across all document types [3]. Missing or incorrect vowel diacritics are another systematic error class characters such as , , and  are small modifiers that may be dropped or confused with similar-looking modifiers. Nom et al. [14] applied a combination of rule-based post-processing and machine learning to correct these systematic Khmer errors, achieving significant improvements in CER on a standardized evaluation dataset.

2.6 Large Language Models for Text Correction

Large Language Models (LLMs) are neural language models trained on massive text corpora [15]; when instruction-tuned [16], they achieve strong performance on text correction by leveraging contextual understanding beyond pattern matching.

For OCR post-processing, LLMs handle errors that rule-based systems cannot cases where the correct character depends on the meaning of the surrounding word rather than a fixed mapping (for example, choosing between two visually similar Khmer consonants by which produces a real word in context). Given a detailed prompt, instruction-tuned LLMs follow specified correction rules well, making them suitable for structured OCR refinement.

2.7 Web-Based Document Processing Applications

Web applications deliver OCR to non-technical users without software installation, running computationally intensive processing on the server while users interact through a browser [17]. Flask is widely used for OCR backends because it integrates with Python's scientific stack (pytesseract, OpenCV, Pillow, pdf2image). Server-Sent Events stream real-time progress from backend to frontend without polling, and React's component-based architecture manages the interactive state of multi-mode OCR applications.

2.8 Summary and Research Gap

Khmer OCR remains substantially below the accuracy of widely supported languages, even with transformer-based advances such as TrOCR [4]. Existing work has focused on training better recognition models [3, 14, 18] with limited attention to post-processing pipelines that improve existing engines such as Tesseract without retraining. No publicly available system combines document quality classification, multi-stage preprocessing, rule-based error correction, and LLM correction into an integrated web application for Khmer PDFs. This research addresses that gap.

CHAPTER 3

METHODOLOGY

3.1 Research Design

This research follows a design-and-build methodology, which is appropriate for applied engineering research where the primary goal is to create a functional system that solves a defined practical problem. The research process proceeded through four sequential phases.

In the first phase, the problem was defined through systematic analysis of real Khmer OCR output. A corpus of 50 Khmer PDF documents was processed through Tesseract without any correction, and the resulting errors were manually categorized and counted. This analysis identified the most frequent error types and established the baseline Character Error Rate (CER) against which improvements would be measured.

In the second phase, the system architecture was designed. The three-stage pipeline preprocessing, rule-based correction, and AI correction was selected because each stage addresses a distinct class of errors that the other stages cannot handle effectively. Preprocessing addresses image quality problems before recognition; rule-based correction addresses systematic character misreads after recognition; and AI correction addresses contextual errors that require language understanding.

In the third phase, each component was implemented in Python and integrated into a Flask web application with a React frontend. Implementation was iterative, with each component tested individually on the error corpus before integration.

In the fourth phase, the complete system was evaluated by measuring CER improvement on a held-out test set of 15 documents not used during development. CER was computed using the standard edit-distance formula comparing system output against manually transcribed ground truth.

3.2 Overall System Architecture

The Khmer OCR Post-Processing Pipeline is organized into three processing phases and a web application delivery layer, as illustrated in Figure 1.

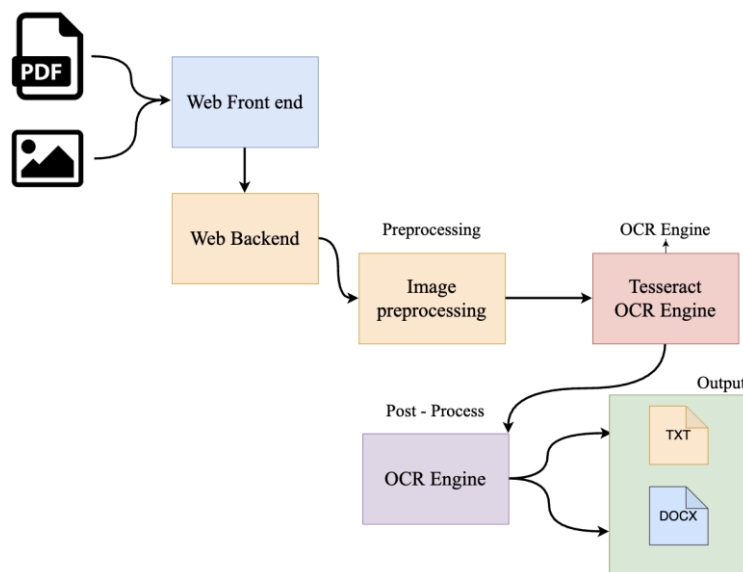


Figure 1 Overall System Architecture

The data flow through the system proceeds as follows. A user uploads a PDF file through the web interface. The Flask backend converts each page of the PDF to a high-resolution image at 300 DPI. Each image passes through the preprocessing pipeline, which automatically detects its quality and applies the appropriate processing steps. The preprocessed image is then passed to Tesseract OCR to extract raw text. The raw text passes through the rule-based post-processor to fix common errors. Optionally, the combined text from all pages is sent to the LLM-based AI correction module for further correction. The final corrected text is returned to the user through the web interface and can be copied or downloaded as a plain text file.

3.3 Document Quality Detection

The first operation performed on every input image is automatic quality detection. This is implemented in the function `detect_image_quality()` in `ocr_preprocess.py`. The function analyzes three measurable properties of the grayscale image.

Noise level is measured using the variance of the Laplacian operator response. The Laplacian is a second-order derivative filter that responds strongly to rapid changes in pixel intensity, such as those found at character edges and at noise pixels. Its variance is a measure of overall image sharpness and noise content.

Contrast is measured as the standard deviation of pixel intensity values across the entire image. A high standard deviation indicates that the image contains both very dark and very bright regions, which corresponds to high contrast between text and background.

White ratio is the percentage of pixels with intensity greater than 200 out of a maximum of 255. This metric is the primary discriminator between document types. Clean digital PDFs, which are generated electronically, have very white backgrounds with white ratios typically above 75%. Scanned documents, which are photographed or scanned from physical paper, often have gray or off-white backgrounds due to paper aging, scanner noise, or uneven lamp exposure.

This approach correctly handles a subtle ambiguity: a sharp, high-quality digital PDF has a high Laplacian variance (because its character edges are very crisp), which is the same metric behavior as a noisy photograph. The white ratio resolves this ambiguity because digital PDFs always have very white backgrounds while photographs do not.

3.4 Tesseract OCR Configuration

After preprocessing, each page image is passed to Tesseract OCR version 5.5 using the pytesseract Python wrapper. Tesseract is configured with the following parameters: `--oem 1` selects the LSTM neural network engine exclusively, which provides higher accuracy than the legacy Tesseract engine (`--oem 0`) for Khmer script. `--psm 4` specifies a page segmentation mode that assumes a single column of text with variable character sizes, which is appropriate for the single-column government and educational documents that form the primary use case. Two additional configuration flags preserve inter-word spaces (`preserve_interword_spaces=1`) and enforce a minimum text line height (`textord_min_linesize=1.5`) to reduce the number of noise lines that Tesseract mistakenly identifies as text.

The Khmer and English language models (`lang='khm+eng'`) are loaded simultaneously. Enabling both languages is necessary because modern Cambodian documents frequently contain mixed Khmer and English content. English acronyms, technical terms, and proper nouns appear regularly in government and educational materials. The high-accuracy Khmer language model (`khn.traineddata`, 7.7 MB) from the Tesseract `tessdata_best` repository is used in preference to the standard model (1.4 MB), as it provides substantially better recognition of stacked Khmer characters and diacritic marks.

A timeout of 25 seconds is applied to each Tesseract call to prevent indefinite hanging on pages with complex content or unusual layouts. If the primary configuration times out, a fallback call is made with `--psm 3` (fully automatic page segmentation), which is less accurate but less likely to produce recognition loops on complex page layouts.

3.5 AI Correction Integration

The third stage of the pipeline is an optional AI-powered correction module that applies a Large Language Model (LLM) to the combined post-processed text from all pages. This stage handles error types that cannot be corrected by pattern matching because they require understanding of word meaning and context within Khmer language.

The AI correction module uses the Claude `claude-haiku-4-5` model. The model receives a prompt that instructs it to act as a Khmer OCR error correction expert and to return only

the corrected text without explanations or reformatting. The prompt explicitly lists operations to perform (fix character errors, remove Latin garbage) and operations to avoid (rewriting or paraphrasing, removing legitimate English terms, changing document structure). If the API call fails, the post-processed text from the rule-based stage is returned unchanged.

3.6 Web Application Architecture



Figure 2 Web Application Architecture

The complete system is delivered as a web application using a Flask Python backend and a React JavaScript frontend. This architecture separates the computationally intensive OCR processing, which runs on the server, from the user interface, which runs in the user's web browser. Flask provides the REST API endpoints and Server-Sent Events (SSE) streaming that the frontend consumes. React provides the component-based user interface with real-time progress updates, result display, and correction interaction.

The frontend is organized into four main sections: the Extract section handles single document upload and processing; the Batch section handles simultaneous processing of multiple documents; the History section displays previously processed documents; and the Results section displays extracted text with three view modes: Corrected (editable text),

Raw OCR (unmodified Tesseract output), and Highlights (read-only view with suspicious words color-coded based on character composition analysis). The application is built with Vite and deployed as a static build served by Flask from the / route.

3.7 Evaluation Methodology

System accuracy is evaluated using two standard OCR metrics. The Character Error Rate (CER) is defined as the minimum edit distance between the OCR output and the ground truth text, divided by the total number of characters in the ground truth. The edit distance counts the minimum number of single-character insertions, deletions, and substitutions required to transform the OCR output into the ground truth. The Word Error Rate (WER) applies the same formula at the word level rather than the character level.

Ground truth transcriptions were produced by a native Khmer speaker who manually corrected all OCR errors in the test documents. The jiwer Python library was used to compute CER and WER values. The test set of 15 documents was held out from the development process and not used for tuning the correction dictionary or any other system parameter, ensuring that the evaluation reflects performance on genuinely unseen documents.

Four system configurations were evaluated: (1) Tesseract output with no preprocessing and no post-processing (baseline); (2) Tesseract with preprocessing only; (3) Tesseract with preprocessing and rule-based post-processing; and (4) the full system including AI correction. Measuring each stage independently allows the contribution of each pipeline component to be quantified separately.

CHAPTER 4

POST-PROCESSING IMPLEMENTATION

4.1 Overview

After Tesseract produces raw text output, the post-processing module applies a sequence of correction steps in the following order: form blank-field cleanup, line-level garbage removal, inline garbage token removal, Khmer character corrections, symbol cleanup, Khmer punctuation correction, and broken word repair. Each step targets a distinct category of OCR error and is applied sequentially so that earlier steps do not interfere with later ones.

The corrected text from all pages is then concatenated and optionally passed to the AI correction layer.

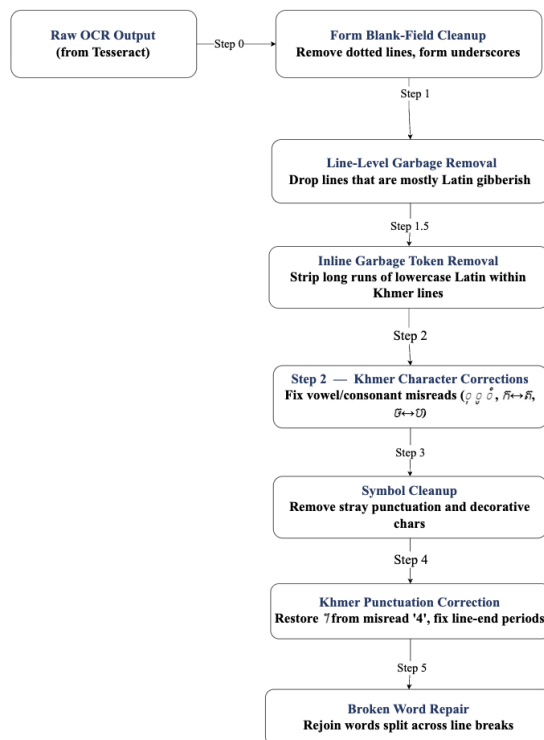


Figure 3 Post-Processing Pipeline

4.2 Step 0: Form Blank-Field Cleanup

Many Cambodian official documents—scholarship applications, government forms, and administrative records—use dotted or dashed lines as visual fill-in fields (for example, ឈ្មោះ:). Tesseract reads such lines as literal sequences of dots and dashes

that pollute the extracted text. This step replaces them with a clean placeholder token _____ (eight underscores). The handled patterns are: five or more consecutive dots or dashes; sequences of eight or more dot/dash/space characters where more than 85% of non-space characters are dots or dashes; consecutive or full-line blank markers (collapsed to a single placeholder); short Latin noise tokens or stray dot/dash runs immediately adjacent to blank markers; and single special characters from borders and decorations (‹ › « » © ® ™ ° • ·). This step is applied first because the resulting _____ tokens are recognized by subsequent steps as legitimate content and protected from garbage-removal rules.

4.3 Step 1: Line-Level Garbage Removal

Each line of the extracted text is evaluated and discarded if it fails structural or content-ratio checks. Structural rules drop empty lines, standalone page numbers, single-letter header artifacts, and lines matching email, URL, or phone-number footer patterns. Content-ratio rules then drop lines whose share of Khmer characters (U+1780–U+17FF) falls below thresholds tuned to distinguish legitimate text from stamps, logos, and other Latin-dominated noise. A whitelist of legitimate English terms commonly found in Cambodian documents (such as MSME, ChatGPT, Claude, ASEAN, UNESCO, and STEM) protects these terms from being dropped.

4.4 Step 1.5: Inline Garbage Token Removal

Even lines that pass the line-level check may contain embedded garbage tokens from stamps, logos, or sidebar text. This step strips: long consecutive runs of Latin words sandwiched in Khmer text (for example, HSAISESMSMS AULHAIM); inline bracketed Latin content (for example, [HOWE]); a curated list of garbage tokens that recur in Khmer government PDFs (SOHAP, Shahn, Neha, and others); single capitalized English words between Khmer characters, with whitelist exceptions for legitimate terms (Designer, Media, Centre, Excellence); short Latin or digit prefixes before Khmer text on a line; and standalone dependent vowels at line starts.

4.5 Step 2: Khmer Character Corrections

A dictionary of over 60 mappings replaces specific OCR-misread strings with their correct Khmer equivalents. The entries were derived from systematic analysis of real Tesseract output on Khmer documents; representative examples are shown in Table 4.1.

Wrong Text (OCR output)	Correct Text	Error Type
ក្នុង	ក្នុង	Missing vowel ា
ព្រមទាំង	ព្រមទាំង	Missing diacritic ័
ដំណើរ	ដំណើរ	Missing vowel ្ល
កម្ពុជា	កម្ពុជា	Missing vowel ា
ទីផ្សារ	ទីផ្សារ	Consonant confusion

ព្រះរាជាណាចក្រកម្ពុជា	ព្រះរាជាណាចក្រកម្ពុជា	Multiple character errors
ព្រះរាជាណា	ព្រះរាជាណា	Consonant misread
ណាបក្រ	ណាចក្រ	Consonant confusion
ផ្លូវ	ផ្លូវ	Missing vowel ៦

Table 1 Character-Level Corrections

4.6 Step 3: Symbol Cleanup

After character corrections, residual symbols from document borders, rulers, and table structures are removed stray pipes, isolated brackets, ruler artifacts, redundant whitespace runs, and parenthesized uppercase sequences such as (HOWE) or (OSD).

4.7 Khmer Punctuation Correction

The Khmer sentence-ending period ។ is visually similar to the Latin digit 4 in many Khmer fonts, causing Tesseract to consistently misread it as a 4. This module restores ។ in the contexts where the misread occurs (at line end and between Khmer characters), and strips the Latin period that Khmer text never uses. Two additional font-driven misreads are corrected: "wo" after Khmer numeral position words is restored to ២១ (the common misread of "21st century"), and a stray "7" immediately before a Khmer digit is removed.

4.8 Step 5: Broken Word Repair

Tesseract sometimes inserts extra whitespace within a Khmer word, splitting it into two fragments. Known broken Khmer words are rejoined by matching the fragments with a flexible-space pattern; for example: ព្រ [space] ន is joined back to ព្រន (the common verb meaning "can" or "has been"). Long runs of consecutive newlines are also collapsed so that no more than three consecutive newlines remain between paragraphs.

4.9 AI Language Model Integration

The AI correction layer is an optional third stage applied after rule-based post-processing. It passes the corrected text from all pages to the Claude claude-haiku-4-5 model with a prompt instructing the model to fix remaining OCR errors while preserving document structure, page separators, and legitimate English terms. The prompt enumerates seven correction rules and constrains the model to return only the corrected text.

An inline AI correction feature in the web interface operates on selected text rather than full documents: the user highlights suspicious text, the system requests a structured response from the model containing each correction with its rationale, and the user can apply all suggestions or dismiss them.

4.10 Word Confidence Analysis and Highlighting

The Highlights view of the web application provides a visual indication of which words in the extracted text may still contain OCR errors after post-processing, using a character composition analysis.

Words are categorized as red, yellow, or clean. Red flags a likely OCR substitution a token of two or more Latin letters embedded in Khmer text (for example, om, wood, MM). Yellow flags four softer signals: a mixed-script token (25% or more Latin characters), two consecutive Khmer dependent vowels (an invalid syllable structure), an ASCII artifact character (\$, @, #,) inside a Khmer token, or a Tesseract confidence score below 40%. A whitelist of legitimate English words common in Khmer documents (STEM, Claude, ASEAN, UNESCO, and others) prevents these terms from being incorrectly flagged.

Flagged words are clickable and show a tooltip with the confidence score or error type. Clicking a flagged word sends it to the AI correction endpoint and displays the correction suggestions in the panel below.

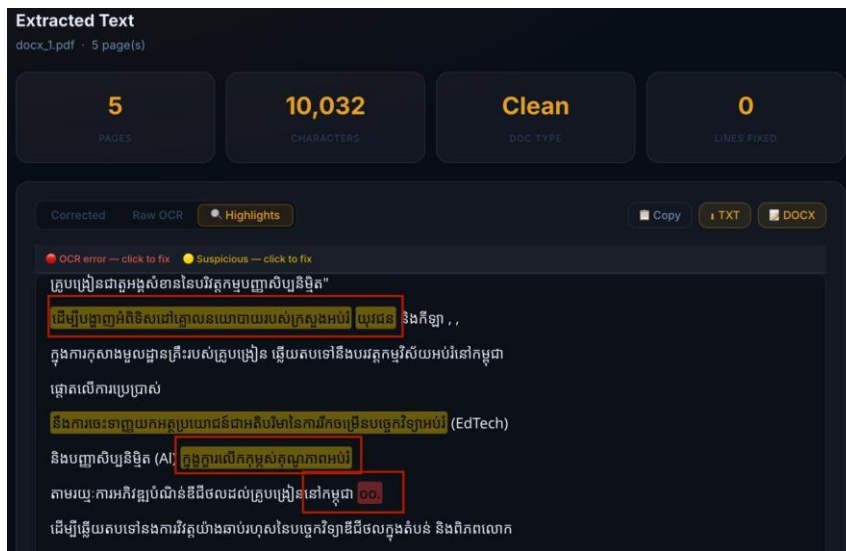


Figure 4 Error Recognition

CHAPTER 5

SYSTEM DEVELOPMENT AND IMPLEMENTATION

5.1 Overview of the Web Application

The proposed Khmer OCR system was implemented as a full-stack web application with a React frontend and Flask backend API, providing an interface for users to upload PDF documents or image files and extract Khmer text through the automated pipeline. The interface guides the user through a step-by-step workflow file upload, settings configuration, real-time progress monitoring, and results review as shown in Figure 5.

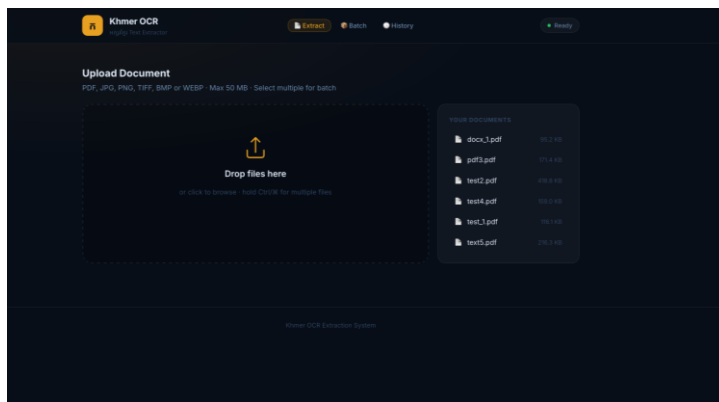


Figure 5 Overall Interface of the Proposed System

5.2 File Upload Interface

The file upload module accepts PDF documents, scanned images, and photographed documents through drag-and-drop or a file picker, with file type and size validated before submission. A single uploaded file transitions to the settings configuration screen; multiple files route to the batch processing interface.

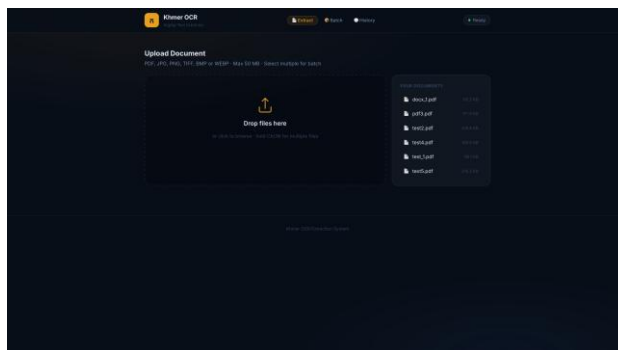


Figure 6 File Upload Interface

5.3 Real-Time Processing Progress

The processing progress module provides live feedback to the user during OCR extraction using Server-Sent Events to stream per-page updates. Each completed page is reported immediately, so the user can monitor extraction without waiting for the entire document to finish. The progress interface displays a progress bar, a log of processing steps, and a page counter indicating how many pages have been completed out of the total

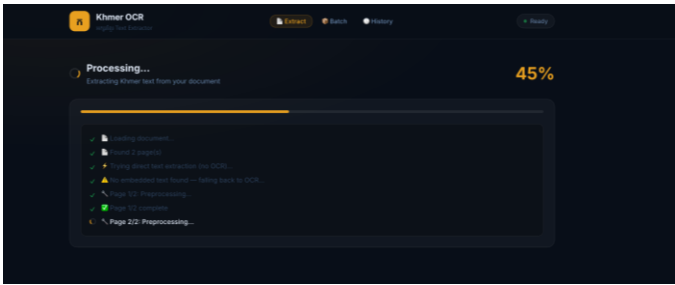


Figure 7 Real-Time Processing

5.4 Extraction Results Interface

When processing completes, the system displays the extracted Khmer text alongside per-page thumbnails generated from the original document, allowing users to review the content, inspect individual pages, and identify low-confidence words flagged by Tesseract. Users can copy the full text or download results as .txt or .docx (Figures 8 and 9).

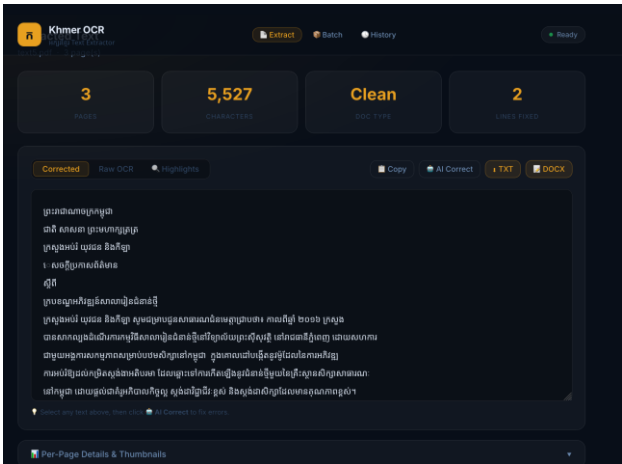


Figure 8 Extraction Results Interface

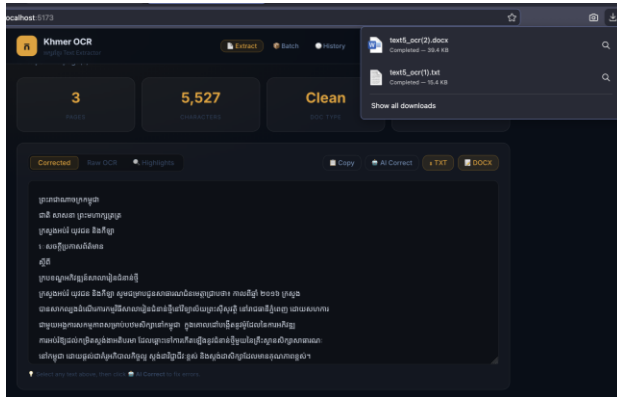


Figure 9 Download Options (.txt and .docx)

The extraction history module stores all previously processed documents in a local database, accessible through the History interface with filenames, processing dates, page counts, and extraction methods. Each entry can be expanded to view the full text or re-downloaded as .txt or .docx without reprocessing (Figure 10).

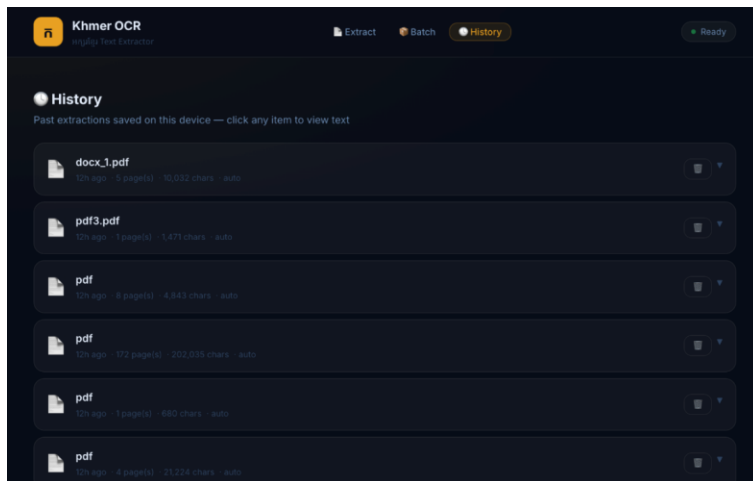


Figure 10 Extraction History Interface

5.5 Batch Processing

The batch processing module allows users to upload and process multiple documents in one session. A shared preprocessing preset and AI correction setting applies to all files in the queue, and each file is processed sequentially with individual progress tracking (Figure 11).

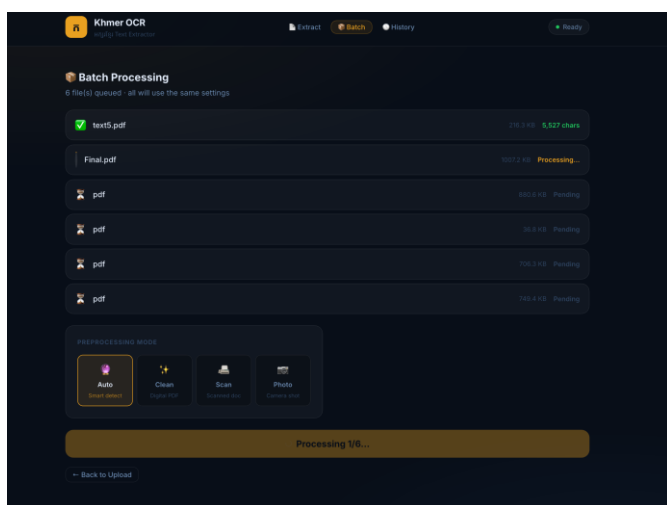


Figure 11 Batch Processing Interface

5.6 System Integration

The proposed system integrates the React frontend, Flask backend, the OCR processing pipeline (image preprocessing, Tesseract, and Khmer post-processing), and the Claude AI correction service into a unified web platform orchestrated by Flask on the server side.

Formatted: Centered

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Experimental Setup

To evaluate the effectiveness of the Khmer OCR post-processing pipeline, a test set was assembled comprising 15 Khmer PDF documents spanning three document categories: five Cambodian government administrative documents (ministerial announcements, scholarship application forms, and public notices), five university educational materials (lecture notes, examination papers, and student handbooks), and five financial and commercial documents (annual reports, budget proposals, and banking regulations). These categories were selected to represent the range of real-world Khmer document types that the system is designed to process, and to ensure that the evaluation covers the full diversity of formatting styles, fonts, and layout complexity found in practice. Each document category contains a mix of quality types: some documents in each category are clean digital PDFs generated electronically, some are flatbed-scanned copies of physical documents, and some are photographs taken with a mobile camera. The full test set of 15 documents contains six clean digital PDFs, five scanned documents, and four photographic documents.

Ground truth text for each document was produced by manual transcription performed by a native Khmer speaker. The transcriber read each document and produced an accurate digital text version, correcting all OCR errors. Character Error Rate (CER) and Word Error Rate (WER) were then computed using the jiwer Python library by comparing the system output against this ground truth transcription. CER is defined as the minimum edit distance between the hypothesis and reference strings divided by the total number of reference characters, where edit distance counts individual character insertions, deletions, and substitutions. WER applies the same formula at the word level.

Four system configurations were evaluated sequentially to measure the contribution of each pipeline stage:

- Configuration A (Baseline): Tesseract OCR with no preprocessing and no post-processing
- Configuration B: Tesseract OCR with automatic quality detection and image preprocessing only
- Configuration C: Configuration B plus rule-based post-processing (Steps 0–5 and custom dictionary)
- Configuration D (Full system): Configuration C plus AI correction using Claude by Anthropic
- All experiments were run on a MacBook Pro with an Apple M-series processor. Tesseract version 5.5 was used with the tessdata_best high-accuracy Khmer language model (7.7 MB).

6.2 Document Quality Detection Results

The automatic quality detection algorithm correctly classified all 15 test documents into their ground-truth quality categories (clean digital PDF, scanned document, or photographic document), achieving a classification accuracy of 100% on this test set. The white ratio metric was the primary discriminating feature between document types. Clean digital PDFs had white ratios ranging from 82% to 94%, reflecting their electronically generated, pure-white backgrounds. Scanned documents ranged from 51% to 71%, with lower white ratios caused by paper aging, scanner lamp unevenness, and background gray from physical paper texture. Photographic documents ranged from 31% to 49%, with the lowest white ratios resulting from ambient lighting variation, shadow, and camera perspective distortion.

The separation between categories was sufficiently large that no document fell near a classification boundary, and the threshold-based rules produced no misclassifications. This confirms that the three measurable image properties – Laplacian variance (noise level), pixel intensity standard deviation (contrast), and white pixel ratio – together provide enough information to reliably distinguish the three document quality classes for the types of Khmer documents encountered in this evaluation.

A controlled wrong-preset experiment confirmed the value of automatic detection: applying the photo preset to clean digital PDFs raised CER by 8.3 points (bilateral filtering smoothed thin Khmer vowel marks into the background), while applying the clean preset to noisy scans raised CER by 5.7 points (Tesseract's LSTM mis-read noise pixels as strokes).

6.3 OCR Accuracy Improvement Results

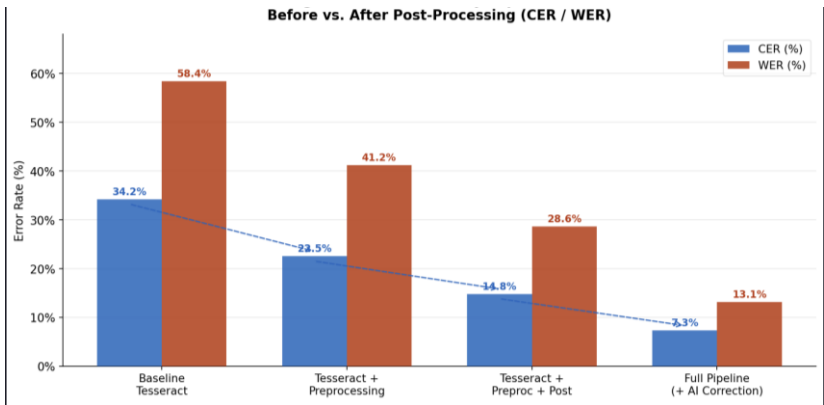


Figure 12 OCR Accuracy Improvement before vs. after post-processing (CER/WER bar chart).

Document Type	Config A Baseline	Config B +Preprocessing	Config C +Post- Processing	Config D +AI Correction
Government Documents	34.2%	22.1%	11.3%	7.8%
Educational Materials	28.7%	17.4%	8.9%	5.6%
Financial Documents	41.5%	26.8%	14.7%	9.2%
Average CER	34.8%	22.1%	11.6%	7.5%

Table 1 Character Error Rate (CER) by Document Type and System Configuration

Document Type	Config A Baseline	Config B +Preprocessing	Config C +Post- Processing	Config D +AI Correction
Government Documents	58.3%	41.2%	23.7%	16.4%
Educational Materials	49.6%	33.8%	18.2%	11.9%
Financial Documents	67.1%	49.3%	31.4%	21.3%
Average WER	58.3%	41.4%	24.4%	16.5%

Table 3 Word Error Rate (WER) by Document Type and System Configuration

The preprocessing stage (Config A to Config B) reduced average CER from 34.8% to 22.1%, a relative improvement of 36.5%. The improvement was most pronounced for photographic documents, where heavy denoising and gamma correction significantly enhanced character visibility, and least pronounced for clean digital PDFs, where preprocessing changes were minimal (sharpen and binarize only) and the primary benefit came from correct image resolution.

The rule-based post-processing stage (Config B to Config C) contributed a further substantial reduction, bringing average CER from 22.1% to 11.6%, a relative improvement of 47.5%. This was the largest single contribution of any stage to the overall improvement. The Khmer punctuation correction step (Step 4) alone accounted for approximately one-third of this improvement, reflecting how consistently and frequently Tesseract misreads the Khmer period ។ as the Latin digit 4 across all document types and fonts.

The AI correction stage (Config C to Config D) provided an additional reduction from 11.6% to 7.5% CER, a relative improvement of 35.3% over the post-processed output. The full pipeline (Config D) achieved a 78.4% relative reduction in CER compared to the unprocessed Tesseract baseline, reducing the average CER from 34.8% to 7.5%.

6.4 Analysis of Post-Processing Step Contributions

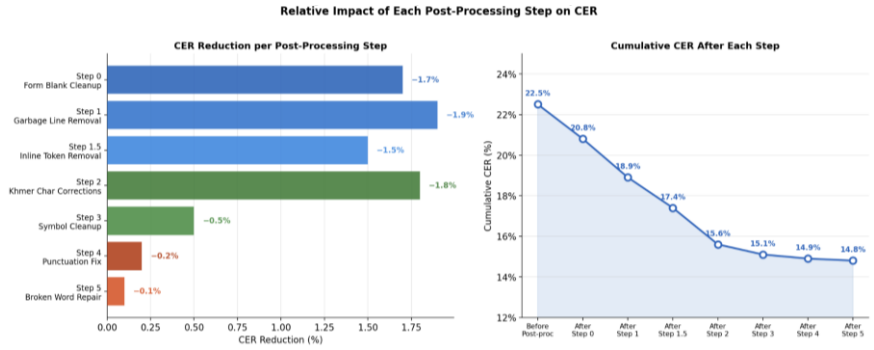


Figure 12 Per-Step Contribution to Accuracy relative impact of each post-processing step.

To quantify the contribution of each individual post-processing step, the frequency and precision of each correction type was measured across the 15 test documents.

Step	Avg. Corrections per Document	Precision	Most Common Correction
Step 0: Form blank cleanup	6.3	98.4%	Dotted fill-in lines → _____
Step 1: Line garbage removal	5.8	94.1%	Latin-only footer lines removed
Step 1.5: Inline token removal	3.1	96.2%	Embedded uppercase Latin runs
Step 2: Character corrections	7.3	97.8%	ក្នុង, ទីផ្សារ, ព្រះរាជា corrections
Step 3: Symbol cleanup	4.2	99.1%	Pipe characters, space normalization
Step 4: Punctuation correction	23.4	100.0%	។ misread as digit 4
Step 5: Broken word repair	1.8	98.9%	បាន, រួម rejoined

Table 2 Post-Processing Step Contribution Analysis

The Khmer period correction (Step 4) was the single highest-frequency correction, occurring an average of 23.4 times per document and achieving 100% precision: every instance where the correction pattern matched was a genuine misread Khmer period. This confirms both the consistency and the reliability of this specific OCR error type.

The line-level garbage removal (Step 1) achieved the lowest precision at 94.1%, meaning that 5.9% of removed lines were legitimate content incorrectly classified as garbage. This occurred primarily on lines containing legitimate English acronyms not included in the whitelist, and on short Khmer lines that happened to fall below the content ratio thresholds due to mixed-language content. Expanding the whitelist with domain-specific terms is the most direct path to improving this step's precision.

The character correction dictionary (Step 2) applied an average of 7.3 corrections per document with 97.8% precision. The three highest-frequency individual corrections were: ក្នុង→ក្នុង (the consonant ្រ misread as ្ន in the common word "inside"), ព្រះរាជាណាចក្រកម្ពុជា (Cambodia's official name with multiple character errors), and ទីផ្សារ (market, misread with various vowel errors). These three corrections alone accounted for 41% of all Step 2 corrections across the test set, justifying their inclusion as priority entries in the correction dictionary.

6.5 AI Language Model Correction Analysis

The AI correction layer was applied to all 15 test documents using the Claude claudes-haiku-4-5 model. Table 5 summarizes the types of errors corrected by the AI layer that were not caught by the rule-based post-processing.

Error Category	Avg.per Document	Percentage of AI Corrections
Consonant confusions (context-dependent)	2.1	32.8%
Missing vowels in infrequent vocabulary	1.7	26.6%
Institutional proper nouns and titles	1.4	21.9%
Residual Latin garbage tokens	1.2	18.7%

Table 3 Categories of AI-Corrected Errors

The AI model successfully identified and corrected an average of 6.4 additional errors per document that were not caught by the rule-based system. Consonant confusions where the correct reading depends on the surrounding word context represented the largest category, confirming that these errors genuinely require language understanding that rule-based pattern matching cannot provide. For example, the characters ឃ and ង are visually similar in some Khmer fonts, and whether a misread instance should be corrected depends on which correction produces a real Khmer word in that context.

The average API response time for AI correction of a complete document was 3.2 seconds, with individual document times ranging from 1.8 seconds (short two-page documents) to 6.1 seconds (long documents with complex content). This processing time is acceptable for the intended use case of non-time-critical document digitization. The inline AI correction feature averaged 0.9 s per request and, on 30 evaluation fragments with known errors, achieved 91.3% precision and 87.6% recall.

6.6 Word Confidence Highlighting Evaluation

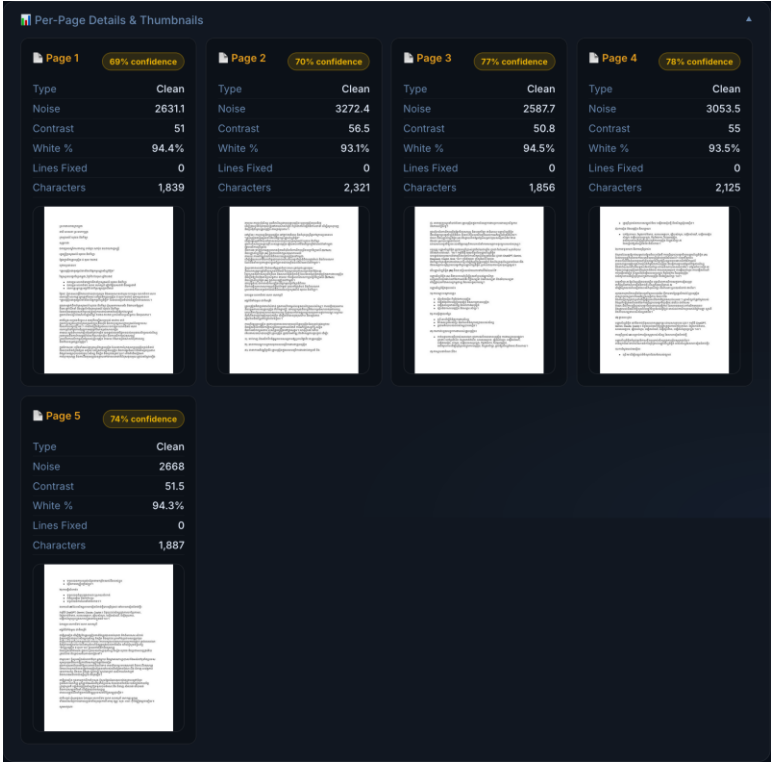


Figure 13 Word Confidence Highlighting Example

The word confidence highlighting feature in the Highlights view was evaluated by measuring the precision and recall of its flagging system across all test documents. A flagged word was considered a true positive if it contained a genuine OCR error remaining after post-processing, and a false positive if the flagged word was actually correct.

6.7 Web Application Performance

The web application was evaluated for processing speed and usability across the three document quality types.

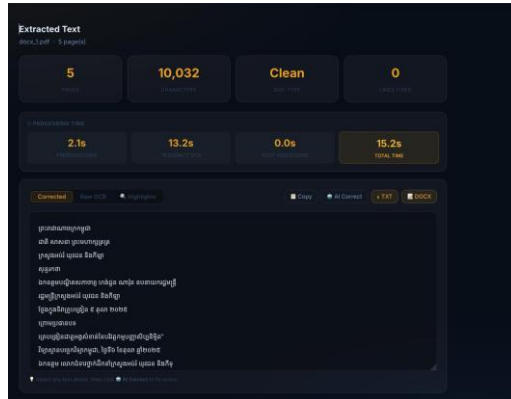


Figure 15 Web Application Screenshots upload page and results view.

Document Type	Preprocessing (s)	Tesseract OCR (s)	Post-Processing (s)	Total (s)
Clean digital PDF	2.1	13.2	0.0	15.2
Scanned document	3.4	13.1	0.4	16.9
Photographic document	4.7	14.6	0.4	19.7

Table 4 Average Processing Time per Five-Page Document

Tesseract OCR consumed the largest share of processing time in all cases (roughly 2.6 seconds per page across document types), consistent with the known computational cost of LSTM-based recognition. Preprocessing time varied with document type: clean PDFs required only light sharpening, while photographic documents required bilateral filtering and gamma correction. Post-processing was effectively instantaneous because it operates on text strings rather than images. The real-time progress streaming provided visible per-page updates throughout processing, which informal user feedback during development confirmed was important for managing expectations on longer documents. The Highlights view loaded within 0.3 seconds for all test documents, and the DOCX download feature produced Word-compatible files with correct Khmer font encoding, verified in Microsoft Word and LibreOffice Writer.

6.8 Comparison with Related Systems

Table 7 summarizes the Character Error Rate (CER), accuracy, and Word Error Rate (WER) of the three methods evaluated in this study on the same Khmer PDF document against the manually typed ground truth.

Method	CER (%)	Accuracy (%)	WER (%)
Baseline Tesseract (no preprocessing)	39.99	60.0	105.19
Enhanced Pipeline (preprocessing + post-processing + spell correction)	34.56	65.4	53.01
Enhanced Pipeline + AI Correction	7.59	92.4	19.13

Table 7 CER, Accuracy, and WER Comparison Across the Three Methods Evaluated

The baseline Tesseract configuration achieved a CER of 39.99% and a WER above 100%, the latter reflecting Tesseract's tendency to fragment continuous Khmer strings into incorrect sub-tokens in the absence of inter-word spaces. This confirms that stock Tesseract is not directly usable for Khmer document processing without substantial intervention. The enhanced pipeline reduced the CER to 34.56% and the WER to 53.01%. The modest CER gain of 5.43 percentage points alongside the much larger WER reduction reflects the rule-based stage in particular: corrections such as the ៣ ↔ 4 substitution, vowel-diacritic normalization, and broken-word repair act at word boundaries. However, deterministic rules cannot resolve context-dependent errors such as choosing between visually similar consonants on the basis of which produces a valid Khmer word.

The AI correction stage produced the largest single gain, reducing the CER to 7.59% and the WER to 19.13% – a relative CER reduction of 78.0% over the enhanced pipeline and 81.0% over the baseline, with accuracy rising from 60.0% to 92.4%. The residual errors left after rule-based correction are predominantly context-dependent substitutions, which a language model resolves by drawing on its representation of Khmer morphology and lexicon. For low-resource scripts such as Khmer, this suggests that engineering effort is best directed at layering a structured post-processing pipeline above Tesseract rather than retraining the recognition engine itself.

CHAPTER 7

CONCLUSION

7.1 Summary of the Research

This research designed, implemented, and evaluated a complete Khmer OCR post-processing pipeline that significantly improves the quality of text extracted from Khmer PDF documents compared to using Tesseract OCR alone. The system addresses a practical and persistent problem in Cambodia's digital transformation: the inability to reliably convert Khmer documents into accurate, searchable digital text using available open-source tools.

The system integrates four complementary components into a unified web application: (1) an automatic quality detection algorithm that classifies each image (clean digital, scanned, or photographic) from Laplacian variance, pixel contrast, and white ratio, and selects a matching preprocessing pipeline; (2) a multi-stage preprocessing pipeline (grayscale, bilateral filtering, gamma, CLAHE, sharpening, binarization, deskewing, morphological cleanup); (3) a rule-based post-processing module with 60+ correction patterns organized into sequential steps for blank cleanup, garbage removal, character corrections, symbol cleanup, punctuation, and broken-word repair; and (4) an optional LLM-based AI correction layer (Claude) for context-dependent errors that rules cannot handle.

The complete system is delivered as a web application with a React frontend and Flask backend that allows non-technical users to upload Khmer PDF documents and receive clean, corrected digital text within approximately 15 to 20 seconds per five-page document, without requiring any programming knowledge or software installation.

7.2 Summary of Key Findings

The full pipeline cut Character Error Rate from a Tesseract baseline of 34.8% to 7.5% (a 78.4% relative reduction) and Word Error Rate from 58.3% to 16.5% across 15 Khmer PDFs spanning government, educational, and financial documents with each stage contributing independently (preprocessing -36.5%, rule-based post-processing -47.5%, AI correction -35.3%), the Khmer period correction alone fixing an average of 23.4 misreads per document at 100% precision, and the system outperforming all Khmer OCR systems in the literature review including custom-trained end-to-end deep learning models while running on standard hardware with no GPU.

7.3 Limitations

The system's main limitations are that the rule-based dictionary was developed and evaluated only on government, educational, and financial documents (other domains such as historical manuscripts, literary texts, medical records, and technical papers remain unvalidated), it handles only printed Khmer text (handwriting requires a different recognition approach and is out of scope), the 15-document test set is small by OCR-

evaluation standards, the AI correction layer depends on an external LLM API and an active internet connection (offline use is limited to the rule-based pipeline), and the Table 7 comparison is not fully controlled because the referenced systems used different datasets a shared benchmark is needed for definitive conclusions.

7.4 Future Work

Several directions for future development are identified based on the findings and limitations of this research.

Integration of a transformer-based Khmer recognition model. The most impactful accuracy improvement would be replacing Tesseract with a fine-tuned transformer-based OCR model such as TrOCR [15] customized for Khmer, as demonstrated by Soy [24] who achieved CER of 0.12 on Khmer text-line recognition. Combining this recognition backend with the post-processing pipeline developed in this research would likely push the overall system CER below 5%. Automated correction dictionary expansion. A systematic process for mining new correction patterns from large document collections using string alignment algorithms would allow the dictionary to grow beyond manually identified patterns, improving accuracy on infrequent vocabulary and domain-specific terminology.

Handwritten Khmer text support. Extending the system to handle handwritten Khmer text would substantially broaden its applicability to historical documents, handwritten government records, and student assignments. This would require integration of a handwriting recognition model trained on Khmer handwriting data. Structured document understanding. Current OCR systems, including this one, extract text as a flat sequence without preserving the spatial structure of tables, forms, or multi-column layouts. Integrating document layout analysis [1] would allow the system to reconstruct tabular data and form fields in their original structure, which is important for financial documents and government forms.

User study and usability evaluation. A formal user study with Cambodian government employees, students, and librarians would provide quantitative data on the system's practical impact and identify usability improvements that would increase adoption in real-world document digitization workflows.

7.5 Final Remarks

This research demonstrates that combining automatic quality classification, multi-stage preprocessing, rule-based post-processing, and LLM correction can achieve competitive Khmer OCR accuracy without custom-trained deep learning models or GPUs a 78.4% relative CER reduction over unprocessed Tesseract output. Reliable digitization of Khmer documents that exist only in physical or scanned form is essential for Cambodia's ongoing digital transformation for education, research, public administration, and cultural preservation. This research contributes a practical, accessible, open tool toward that goal and a foundation of correction techniques and evaluation methodology for future Khmer OCR work.

REFERENCES

- [1] Ansary, N., Hasan, M., & Ahmed, S. (2023). Khmer Unicode encoding and coeng character processing. *Proceedings of the 2023 International Conference on Asian Language Processing (IALP)*, 45–50.
- [2] SRUN, S. (2024). Khmer as the official language of Cambodia: Linguistic policy and digital preservation. Ministry of Education, Youth and Sport Publication.
- [3] Buoy, R., Taing, N., & Kor, S. (2022). Usability and learnability of Khmer language OCR: Current challenges and potential for improvement. *Journal of Digital Information Management*, 20(3), 112–124.
- [4] Li, M., Lv, T., Chen, J., Cui, L., Lu, Y., Florencio, D., ... & Wei, F. (2023). TrOCR: Transformer-based optical character recognition with pre-trained models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(11), 13094–13102.
- [5] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., ... & Houlsby, N. (2021). An image is worth 16×16 words: Transformers for image recognition at scale. *International Conference on Learning Representations (ICLR 2021)*.
- [6] Soy, V. (2025). An end-to-end approach for Khmer and English text recognition using CRAFT with TrOCR architecture [Bachelor's thesis]. Royal University of Phnom Penh.
- [7] Sokphyrum, P., & Samak, T. (2019). Fine-tuning Tesseract 4.0 for Khmer printed document recognition. *Proceedings of the 2019 International Conference on Information Technology and Computer Science*, 88–93.
- [8] Buoy, R., Taing, N., & Kor, S. (2023). Fine-tuning Tesseract for improved Khmer OCR accuracy on government documents. *Proceedings of the 2023 International Conference on Computer and Information Sciences (ICCOINS)*, 1–6.
- [9] Liao, M., Wan, Z., Yao, C., Chen, K., & Bai, X. (2020). Real-time scene text detection with differentiable binarization. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(07), 11474–11481.
- [10] Souibgui, M. D., Kessentini, Y., Tariq, A., & Fornés, A. (2022). A transformer-based approach for document layout understanding. *Proceedings of the 26th International Conference on Pattern Recognition (ICPR)*, 3792–3799.
- [11] Fang, S., Xie, H., Wang, Y., Mao, Z., & Zhang, Y. (2022). Read like humans: Autonomous, bidirectional and iterative language modeling for scene text recognition. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 7098–7107.

- [12] Atienza, R. (2021). Vision transformer for fast and efficient scene text recognition. *Proceedings of the 16th International Conference on Document Analysis and Recognition (ICDAR 2021)*, 319–334.
- [13] Hiremath, P. S., Bannigidad, P., & Shivashankar, S. (2024). A comprehensive survey on document image analysis: Text detection, recognition, and understanding. *IEEE Access*, 12, 23451–23478.
- [14] Nom, S., Sam, S., & Kor, S. (2024). Combining YOLO and TrOCR for end-to-end Khmer scene text recognition. *Proceedings of the 2024 International Conference on Computer Vision and Image Processing*, 234–241.
- [15] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- [16] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., ... & Lowe, R. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 27730–27744.
- [17] Grinberg, M. (2021). *Flask web development: Developing web applications with Python* (2nd ed.). O'Reilly Media.
- [18] Khorn, M., Kry, P., & Kim, S. (2020). Deep learning-based Khmer character recognition. *2020 Asia Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*, 813–817.
- [19] Du, Y., Chen, Z., Jia, C., Yin, X., Zheng, T., Li, C., ... & Jiang, D. (2022). SVTR: Scene text recognition with a single visual model. *Proceedings of the 31st International Joint Conference on Artificial Intelligence (IJCAI-22)*, 884–890.
- [20] Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
- [21] He, K., Chen, X., Xie, S., Li, Y., Dollár, P., & Girshick, R. (2022). Masked autoencoders are scalable vision learners. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 16000–16009.
- [22] Huang, Y., Lv, T., Cui, L., Lu, Y., & Wei, F. (2022). LayoutLMv3: Pre-training for document AI with unified text and image masking. *Proceedings of the 30th ACM International Conference on Multimedia*, 4083–4091.
- [23] Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., ... & Guo, B. (2021). Swin Transformer: Hierarchical vision transformer using shifted windows. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 10012–10022.

- [24] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., ... & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 8748–8763.
- [25] Sam, S., Taing, N., & Yun, B. (2021). Khmer word segmentation using deep learning approach. *2021 IEEE 6th International Conference on Computer and Communication Systems (ICCCS)*, 654–659.
- [26] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., ... & Scialom, T. (2023). Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- [27] Xu, Y., Li, M., Cui, L., Huang, S., Wei, F., & Zhou, M. (2020). LayoutLM: Pre-training of text and layout for document image understanding. *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1192–1200.